Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота № 2

з дисципліни «Системне програмування»

на тему «Часові потоки LightBench: монтаж, статистики й експорт»

Виконав:
студент групи ІО-32
Пашко Максим Ігорович

Перевірив:
Гордієнко Нікіта Юрійович

Київ – 2026

Тема: Часові потоки LightBench: монтаж, придатність пар, статистики й експорт.

Мета роботи: Інтегрувати три компоненти з різними інтерфейсами, перевірити одночасний збір і точки оператора, дослідити власні записи логічного аналізатора та протестувати переносні шари прошивки.

Постановка завдання

1. Обґрунтувати пінові призначення, тактування, підтяжки й бюджет живлення.
2. Реалізувати незалежні 20-мс потоки raw і положення, часові критерії пари та debounce кнопки.
3. Додати статистики серії, кільце 64 точок і сталий CSV-знімок із CRC32.
4. Перевірити I²C, UART, GPIO-маркери ADC/IMU й негативний сценарій NACK.
5. Протестувати App/Devices і runtime без HAL, використати власні байти burst як fixture.

Ця робота реалізує FR-01–FR-11 і FR-13–FR-14 з PRD. FR-12 є завданням ЛР3. Raw не перетворюється на точні люкси; кут від тяжіння призначений для близького до статичного стану.

Хід роботи

Аргументація монтажу

LightBench є стендом порівняння світла й положення у вибраних оператором точках. Форма B використовує STM32 HAL, Nucleo-C031C6 [1] та три компоненти з різними інтерфейсами: MPU6050 через I²C, LDR через ADC, кнопку через GPIO. Камера, SPI-пристрій і силовий виконавчий вузол до поточної схеми не входять.

PB8/PB9 мають AF6 I²C1 і виходять на D15/D14. PA0 підтримує ADC1 IN0. PA1 обрано окремо від analog входу; він має pulldown, кнопка з’єднує його з 3V3. PA2/PA3 використовують AF1 USART2, а PA5 бортовий LED. Підстави наведені у DS13867, §4, та UM2953, рис. 7, табл. 11 [2, 3]. PA13/PA14 збережено для SWD. Позначення Arduino D0/D1 не замінюють USART2; змішування цих назв дало б неправильний UART.

MCU	Роз’єм	З’єднання та аналізатор
PB8 AF6	D15, CN5.10	MPU SCL, 4.7 кОм до 3V3, D0 SCL
PB9 AF6	D14, CN5.9	MPU SDA, 4.7 кОм до 3V3, D1 SDA
PA0 IN0	A0, CN8.1	LDR AO, 0..3.3 В
PA1 GPIO	A1, CN8.2	Кнопка до 3V3, input pulldown, D3 BUTTON
PA2 AF1	VCP, CN7.37	Serial RX, D2 UART_TX
PA3 AF1	VCP, CN10.10	Serial TX
PA4 GPIO	A2, CN8.3	D4 ADC_MARK
PA5 GPIO	D13, CN5.6	LED, D5 LED
PB0 GPIO	D10, CN5.3	D6 POINT, toggle на кожній точці
PB1 GPIO	A3, CN8.4	D7 IMU_MARK під час burst
3V3, GND	CN6.4/6	MPU, LDR, кнопка, спільний GND аналізатора

Інструкція підключення й бюджет живлення

При вимкненому живленні спочатку з’єднуються GND усіх вузлів. MPU VCC і LDR VCC підключаються до 3V3. AD0 MPU підключається до GND, що задає 0x68. INT/XDA/XCL та LDR DO лишаються вільними. Після цього підключаються SCL/SDA з підтяжками, AO до PA0 та кнопка між PA1 і 3V3. На фізичному модулі слід перевірити вже наявні підтяжки, щоб не додати зайве паралельне навантаження.

TX MCU підключається до RX монітора, RX MCU до TX. Аналізатор підключається за таблицею; його канали є входами. Після запуску `selftest` має показати ADC=1 та IMU=1. `run` починає серію і вмикає LED; натискання M або кнопки дає одну точку. Схема `diagram.json` є єдиним монтажним джерелом моделі.

MPU6050 має typical 3.8 мА для accel+gyro за TDK Product Specification, §6 [4]. Це не maximum струм довільного breakout. Для проєктного бюджету прийнято MPU 5 мА, LDR 5 мА, LED 5 мА, кнопку з pulldown менше 0.1 мА та дві підтяжки до 1.4 мА. Сума менше 17 мА, резерв для зовнішніх вузлів 25 мА. Для плати прийнято проєктні 100 мА. За лімітом 3V3 500 мА з UM2953, §6.5.2, плановий залишок становить 375 мА.

Це розрахунок, не вимір споживання. Піки струму плати та конкретного модуля не підтверджені. LDR AO не повинен перевищувати VDDA; живлення невідомого модуля від 5 В без перевірки AO неприпустиме. Силових навантажень немає.

При LOW одна підтяжка споживає 3.3/4700=0.70 мА. За припущення C=100 пФ оцінка часу наростання 0.8473RC=0.40 мкс, менше 1 мкс standard mode. Реальні ємність і фронти треба перевіряти на стенді. Цифрова модель цього не доводить.

Альтернативи відхилено з конкретних причин. LDR і кнопка на PA1 конфліктують за режимом. SPI1 SCK на PA5 зайняв би LED. Внутрішня flash для майбутньої FR-12 не потребує проводів. Зовнішня SPI NOR як запасний варіант потребуватиме нової схеми та перевірки CS/SCK/MISO/MOSI.

Тактування і драйвер MPU6050

HSI48 дає SYSCLK/HCLK/PCLK 48 МГц без дільників. Для I²C встановлено TIMINGR дорівнює `0x20303E5D`: PRESC=2, SCLDEL=3, SDADEL=0, SCLH=62, SCLL=93. Сума high+low дорівнює 157 тактам по 3/48 МГц, тобто 9.8125 мкс без фільтрів і фронтів. Ціль 100 кГц, у VCD медіана періоду 10 мкс.

Адреса MPU є 7-бітним значенням 0x68. На дроті write/read мають байти 0xD0/0xD1. Ініціалізація читає WHO_AM_I, 0x75 має повернути 0x68. Далі записуються 0x6B←0x01, 0x19←0x13, 0x1A←0x03, 0x1B←0x00 та 0x1C←0x00. За register map [5] DLPF=3 і divider=19 задають 1 кГц/20=50 Гц; діапазон accel ±2g, 16384 LSB/g. Gyro ±250°/c та температура декодуються драйвером як супутні поля, але кут обчислюється з accel.

```
BusResult mpu6050_read(Mpu6050 *device,
                       ImuSample *sample) {
    uint8_t bytes[14];
    BusResult result = device->bus.read(
        device->bus.context, device->address,
        0x3b, bytes, sizeof(bytes));
    if (result == BUS_OK)
        mpu6050_decode(bytes, sample);
    return result;
}
```

SensorBus містить context і callback read/write. Драйвер не включає HAL; у runtime callback реалізує board, у host-тесті fake bus перевіряє адресу, регістр, довжину та

порядок операцій. На помилці вихідний sample не змінюється.

Порт I²C використовує AUTOEND із двома циклами pointer/read, короткий settle 120 мкс та спільний timeout 10 мс. Це адаптація існуючого власного порту для моделі. Після NACK модель показує додатковий START без адреси. Така операція лишається помилкою, а ADC продовжує працювати. Повтор ініціалізації обмежено інтервалом 1000 мс.

Шари та компонентна діаграма

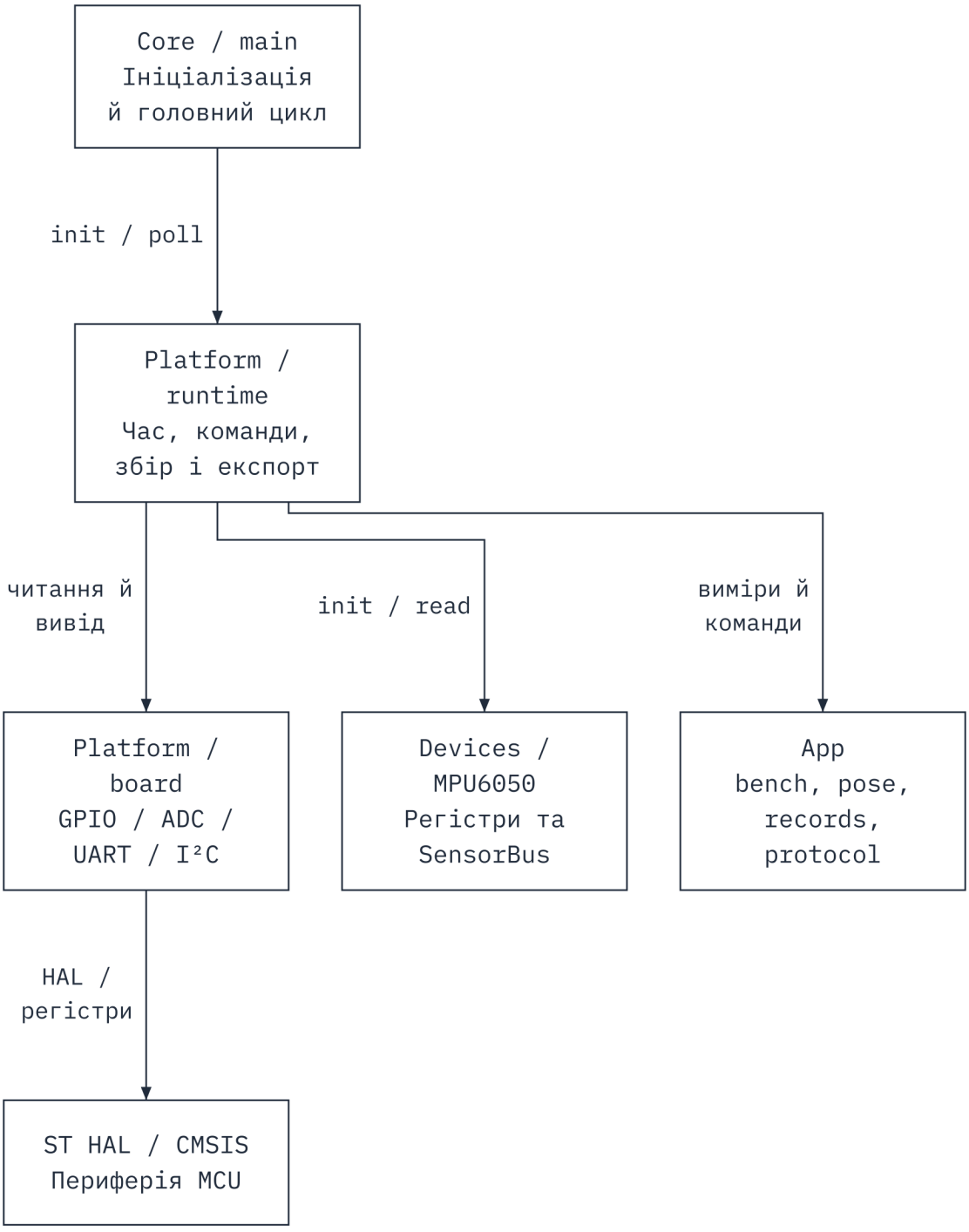


Рисунок 1 – Огляд залежностей модулів LightBench, згенерований у Flowcraft

Стрілка йде від споживача до залежності. Core/main викликає runtime; runtime використовує board, MPU та App; board звертається до HAL. Діаграма є оглядом модулів, не повним графом викликів. Core також ініціалізує HAL. SensorBus передає доступ до

board у драйвер через callback.

App не знає GPIO, UART або I²C. `make check` перевіряє заборонені залежності та SHA256 vendor-файлів. HAL ADC потрібний для PA0, UART для IT RX/TX, GPIO для кнопки/LED/маркерів, I²C для конфігурації шини. RCC/PWR/FLASH/CORTEX забезпечують запуск і clock. Заголовки DMA/EXTI збережено в базовій конфігурації, хоча кнопка опитується й DMA не використовується. Linker GC/LTO прибирає недосяжний код. Це не реалізація DMA-збору.

Часова модель та серія

LightSample і PoseSample мають власний час завершення читання. Збір виконується незалежно з цільовим періодом 20 мс. Кнопка бере останні завершені дані, не запускає додатковий синхронний вимір. Пара придатна, якщо кожному джерелу не більше 40 мс, розрив між часами не більше 20 мс, обидва канали справні й кут придатний.

$$\theta = \arccos\left(\frac{a_z}{\sqrt{a_x^2 + a_y^2 + a_z^2}}\right), \qquad 0^\circ \le \theta \le 180^\circ.$$

Кут обчислюється лише за $0.85 \le |a| \le 1.15g$. Нульовий та динамічний вектори непридатні. Значення 65535 є sentinel, а не кутом. Таке правило не гарантує відсутності повільного додаткового прискорення; оцінка призначена для близького до статичного положення.

Прапорці точки є бітовою маскою: LIGHT_BAD=1, POSE_BAD=2, LIGHT_OLD=4, POSE_OLD=8, SKEW=16, DYNAMIC=32. Відхилена пара теж записується, щоб не приховувати якість збору. Uptime uint32 у мс не є календарем. Різниці витримують один wrap для інтервалів менше 2^{31} мс.

`run` за справного ADC починає нову серію і скидає її статистики; повторний run відхиляється. Stop виконується один раз. ADC fault завершує серію, IMU fault її не зупиняє. Максимальна тривалість серії 3600000 мс. Raw sum і angle sum мають uint64 та різні знаменники. Непридатні кути не входять у mean angle. Немає кутів означає invalid, порожня серія означає empty.

Debounce перевіряє стабільність стану 30 мс для натискання й відпускання. Тривале

утримання не повторює запис. Ціль після підтвердження натискання до 50 мс, разом до 80 мс. Час точки фіксує підтвердження, edge_ms зберігає початок стабільного кандидата.

RAM-журнал та експорт

ReferencePoint займає 28 байтів: п’ять uint32 для sequence, series та трьох часів; чотири uint16 для raw, angle, flags і reserved. Кільце зберігає 64 останні точки, overwritten рахує витіснення. Сталий знімок має ще 64 записи; разом масиви займають 3584 байти. Новий run не стирає попередні точки.

`export` копіює журнал у сталий знімок. Runtime передає по одному CSV-рядку, якщо у TX є достатньо місця. CRC обчислюється на цьому самому кроці для одного запису. Збір і кнопка продовжують працювати; нова точка не змінює вже створений знімок. Повторний export дає busy.

CRC32 IEEE використовує polynomial 0xEDB88320 та початковий/кінцевий XOR 0xFFFFFFFF. Канонічні байти мають little-endian порядок, формат п’ять uint32 і чотири uint16. C padding не хешується. Незалежна Python-перевірка використовує `struct.pack("<IIIIHHHH", ...)` та zlib.crc32, перевіряє count, порядок і діапазони полів. Перемішані рядки status/point не стають CSV.

Фінальний перший знімок містить seq 10..73, count=64, overwritten=9, CRC a391cbd3. Точка 74 з’явилася під час передачі, але не потрапила до цього знімка. Після NACK другий експорт має overwritten=11, одну непридатну пару та CRC 1d5afb21. Обидві перевірки завершилися успішно.

UART має TX-кільце 2048 і RX-кільце 256 байтів. Обробка входу обмежена 64 символами за poll; командний рядок до 47 символів. Невідомі команди й переповнення відхиляються до кінця рядка. RX/TX lost рахуються окремо. Передача через IRQ не усуває очікування вже поставлених байтів; status під час export може затримуватися. Окремий wire-вимір цілі status≤100 мс без черги не виконувався.

Сховище позначено `ram_volatile`. CRC експорту не забезпечує відновлення після reset. FR-12 ще не реалізовано. Для ЛР3 зарезервовано останні дві сторінки внутрішньої

flash, по 2 КіБ.

Аналіз I²C: ідентифікація й burst

Модель аналізатора описана в документації Wokwi [6]. Захвати зроблено у власному запуску 27.10.2026. Повний VCD має 29.25814355 с модельного часу з паузами для зміни входів. Вікна походять з нього без синтезу сигналів; origin зберігає offset та SHA256. SR має 20 МГц, крок 50 нс, що дає близько 200 відліків на період SCL. Для бітового декодування потрібно щонайменше декілька відліків на найкоротший стан; 20 МГц має достатній запас для цих 100 кГц і UART 115200.

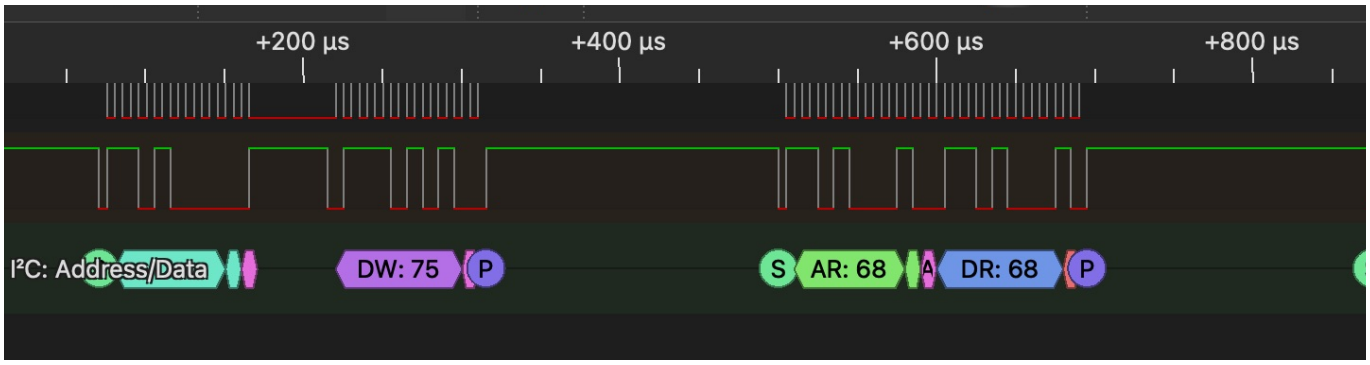


Рисунок 2 – PulseView: pointer 0x75 і читання WHO_AM_I=0x68

Ручна перевірка без декодера: SDA падає при високому SCL, це START. На восьми наступних фронтах SCL SDA має 11010000, байт 0xD0. Старші сім бітів дають адресу 0x68, R/W=0. Дев'ятий LOW є ACK. Далі 01110101 означає регістр 0x75, за ним ACK. Після STOP і нового START байт 11010001 означає read; дані 01101000 дорівнюють 0x68. Master завершує останній байт NACK і STOP.

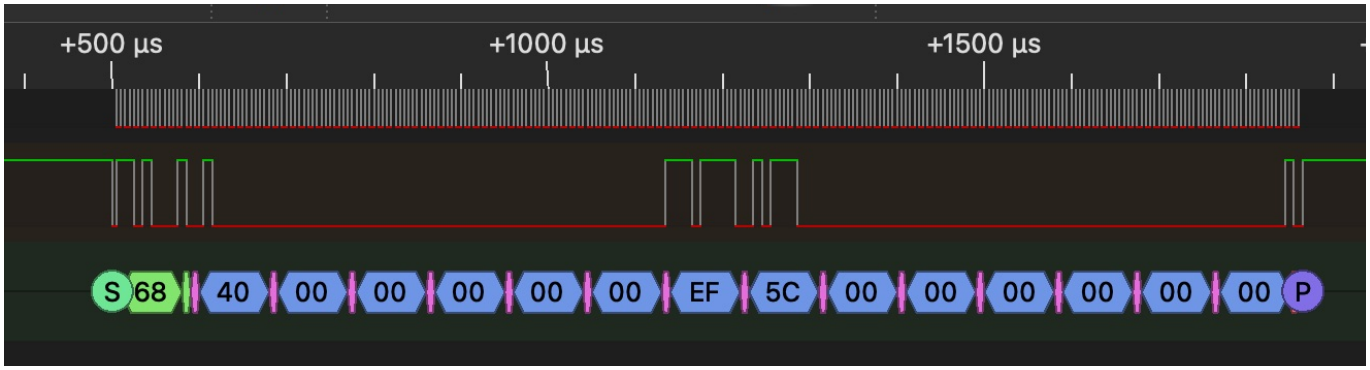


Рисунок 3 – Читання 14 байтів accel/temp/gyro із 0x3B

Read-фаза WHO_AM_I займає 195 мкс, pointer+read 624.275 мкс. Для burst відповідно 1365 і 1899.375 мкс. На read-фазі 15 байтів із адресою дають 135 тактів, близько 1.35 мс за 100 кГц; START/STOP та паузи пояснюють решту.

Байти обраного burst починаються 40 00 00 00 00 00. Signed big-endian дає (16384,0,0), після масштабу (1000,0,0) mg і кут 90°. Цей запис використано як fixture драйвера. Температурні байти EF 5C дають 24001 m°C; це значення моделі, не вимір реального термометра.

Аномалія NACK і відновлення

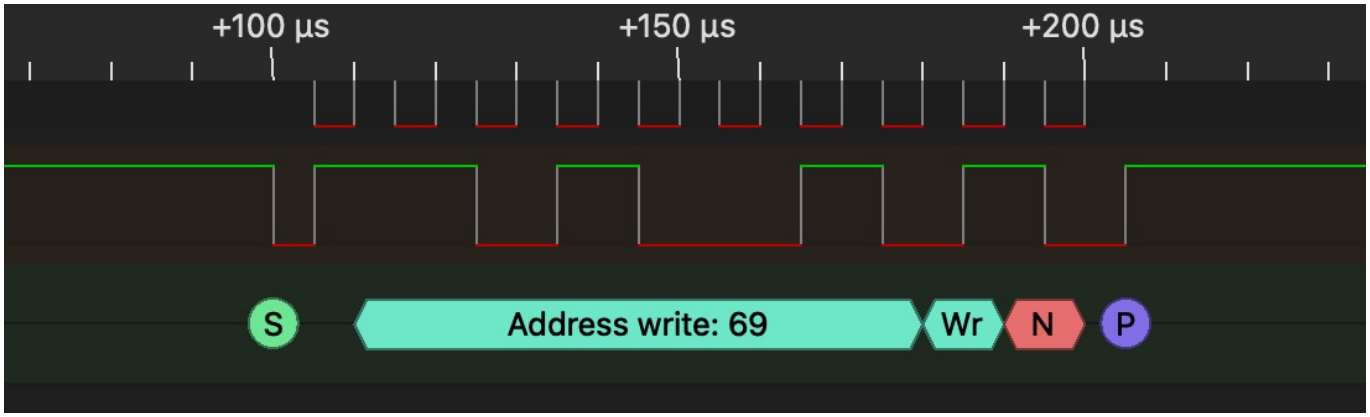


Рисунок 4 – Неправильна адреса 0x69, NACK і додатковий START

За AD0=0 команда `imuaddr 0x69` спрямувала звернення на відсутню адресу. На дроті 0xD2, біт ACK HIGH. Операція займає 105 мкс; додатковий START без повного байта належить адаптеру моделі після помилки. Декодер і первинна форма сигналу показують невдале звернення.

Проби ініціалізації відбулися у ms 26271 і 27271, тобто через 1000 мс. У цей час ADC продовжив збір. Точка 75 має raw=32, angle=65535 і flags=26. Це POSE_BAD, POSE_OLD та SKEW, а не справна пара. Після повернення адреси 0x68 selftest у ms 27832 показав ADC=1 та IMU=1.

Результат серії	Значення після stop у ms 27833
Кількості	ADC 1382, pose 1304, придатних кутів 1304
Raw	mean=129, min=32, max=3413
Кут	mean=8523 сотих градуса
Якість	ADC fail=0/gap=0; IMU fail=2/gap=1
UART	RX lost=0, TX lost=0

Gap означає інтервал понад 40 мс, failure означає невдалу спробу. Дві помилки та одна gap не суперечать одна одній. Граничний timeout і ADC fault перевірено fake-тестами; фізичне відключення АО не виконувалося.

ADC, GPIO і UART

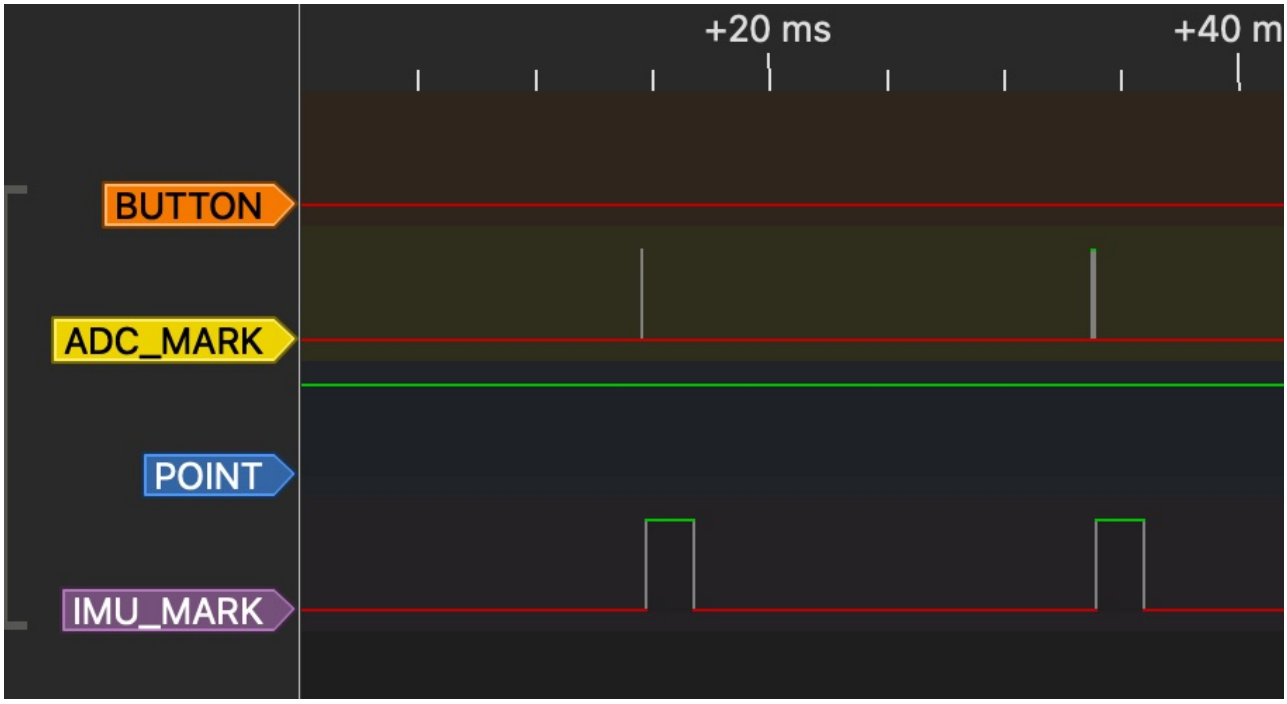


Рисунок 5 – ADC_MARK та IMU_MARK у періодичному зборі

ADC працює при PCLK/2=24 МГц. Теоретичні 160.5+12.5 циклів дають 7.21 мкс. Медіана ширини ADC_MARK 50.725 мкс охоплює HAL start/read/stop, тому не дорівнює лише aperture. Медіана періоду 19.996825 мс, найбільший 27.5046 мс; інтервалів понад 40 мс немає. Додатковий selftest дає коротший мінімальний інтервал 9.5134 мс.

IMU_MARK має медіану ширини 2.0147 мс та періоду 19.993575 мс. Максимальні

1577.638 мс належать навмисному NACK і відновленню. BUTTON містить 7457 сирих імпульсів у 75 групах, POINT має 75 змін. До маркера точки 29.623..39.167 мс. Мінімум нижче 30 мс на частку мілісекунди через квантування SysTick; програмна перевірка elapsed≥30 виконується в мс. Максимум нижче цілі 80 мс.

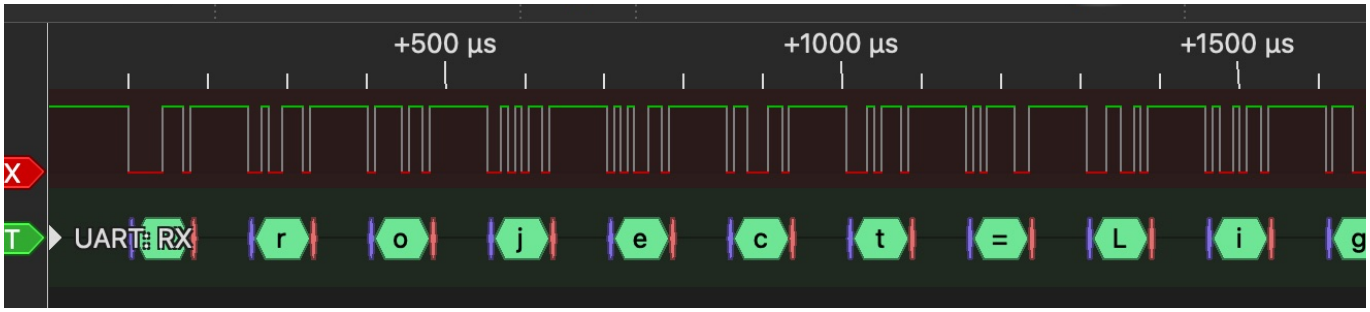


Рисунок 6 – UART 115200 8N1, початок банера project=LightBench

Розрахунковий біт 1/115200=8.6806 мкс. Медіана переходів одного біта 8.7 мкс, розбіжність близько 0.22%. Кадр має start, 8 data LSB-first, stop, без parity. Декодований банер збігається з UART-журналом. Роздільність SR 50 нс також обмежує точність цього порівняння.

Тестування та ресурси

У macOS і Linux пройшли чотири незалежні C-набори та три Python-тести. ASan/UBSan пройшли на обох платформах. Fake bus є керованою реалізацією інтерфейсу, а перевірки очікуваних регістрів і порядку операцій виконують роль mock-очікувань.

Набір	Перевірені властивості
bench	181 кут, межі норми, 40/41 і 20/21 мс, wrap, статистики з окремими знаменниками, fault/stop, debounce, кільце, CRC
protocol	Точні команди, CRLF, невідомі/довгі рядки, overflow recovery
mpu	Конфігурація, signed scale, ID/NACK/timeout, незмінність виходу, власний burst fixture
runtime	Одночасний збір, кнопка під час export, retry 1 с, ADC stop, відновлення, годинний ліміт через fake clock
Python export	Незалежний CRC, count/порядок/діапазони, обірвані й пошкоджені дані

Coverage App+Devices становить 170/170 рядків, 126/130 гілок, 96.92%. Board/HAL та runtime не входять до цієї метрики. Непройдені гілки pose містять захисні межі арифметики; метрика не замінює верифікацію заліза.

Дві чисті збірки дали однакові ELF/HEX/BIN. Flash 23936 із 28672 байтів; RAM за map 8456 із 12288, включно з резервами heap/stack. Окремий найбільший стековий кадр 408 байтів не є піком сумарного стеку. Профайлінг піка, reset-сховище й годинний baseline залишаються ЛР3. Прискорений fake clock не замінює годинного запуску.

CI-файл налаштований, але віддалений запуск цієї версії не заявляється. Реальні захвати і код фіксуються SHA256; перевірений банер ed8ba61-d5b4dbed відповідає parent commit плюс checksum, а не твердженню про чистий tree на час запису.

Журнал проблем і виправлень

1. Flash overflow. Причина: код і бібліотечні helpers перевищили 28 КіБ. Виправлення: LTO і bounded conversion; результат 23936 байтів.
2. Зайві double helpers. Причина: int64→float на M0+. Виправлення: перевірка діапазону, потім uint32→float.
3. Пропуск ADC під час export. Причина: CRC 64 записів обчислювався синхронно перед передачею. Виправлення: CRC по одному рядку; фінальний VCD не містить ADC gap понад 40 мс.

4. Неінформативний build=local. Причина: відсутність git у контейнері. Виправлення: Makefile передає parent hash та checksum джерел.
5. Попередження LTO listing. Причина: per-file assembler listing. Виправлення: прибрано несумісні параметри; два build log без warning.
6. Некоректні підписи PB0/PB1 у pinout. Причина: змішано нумерацію MCU й Arduino. Виправлення: PB0=D10, PB1=A3 за UM2953; перевірено схему й GPIO.

Код і перевірки виправлень 1–5 містить коміт `6469e71`; підсумкова документація фіксує виправлену таблицю з п. 6. Старий прототип із синхронним CRC не є джерелом фінальних вимірів. Дата фактичного виконання всіх нових вимірів 27.10.2026.

Висновки

У LightBench інтегровано LDR, MPU6050 і кнопку з різними інтерфейсами. Кожен потік має власний час; точка зберігає обидва джерела та причини непридатності. Статистики відокремлюють кількості raw і придатних кутів. RAM-кільце й сталий знімок дозволяють експортувати дані під час збору.

Власні записи підтвердили 100 кГц I²C, близько 20 мс для двох потоків, одну точку на натискання та відновлення після NACK без зупинки ADC. Незалежна перевірка двох CSV підтвердила count і CRC. Host-тести й санітизатори пройшли; portable coverage 100% рядків і 96.92% гілок.

Симуляція не підтверджує фізичну точність, струм і фронти сигналів. FR-12 після reset не виконано, flash лише зарезервовано. Wire latency status та пік стеку окремо не виміряні.

Код, PRD, монтаж і manifest наведено у репозиторії LightBench [7]; поточна версія перевірена локально.

Список використаних джерел

[1] Wokwi. *STM32 Nucleo64 C031C6*. URL: https://docs.wokwi.com/parts/board-st-nucleo-c031c6 (дата зверн. 27.10.2026).

[2] STMicroelectronics. *STM32C031x4/x6, DS13867, Rev. 4*. URL: https://www.st.com/resource/en/datasheet/stm32c031c6.pdf (дата зверн. 27.10.2026).

[3] STMicroelectronics. *STM32 Nucleo-64 MB1717, UM2953*. URL: https://www.st.com/resource/en/user_manual/um2953-stm32-nucleo64-board-mb1717-stmicroelectronics.pdf (дата зверн. 27.10.2026).

[4] TDK InvenSense. *MPU-6000 and MPU-6050 Product Specification*. URL: https://product.tdk.com/system/files/dam/doc/product/sensor/mortion-inertial/imu/data_sheet/mpu-6000-datasheet1.pdf (дата зверн. 27.10.2026).

[5] TDK InvenSense. *MPU-6000 and MPU-6050 Register Map, Rev. 4.2*. URL: https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Register-Map.pdf (дата зверн. 27.10.2026).

[6] Wokwi. *Wokwi Logic Analyzer*. URL: https://docs.wokwi.com/parts/wokwi-logic-analyzer (дата зверн. 27.10.2026).

[7] Th1sMaxOn. *LightBench: firmware, PRD, pinout and captures*. URL: https://github.com/Th1sMaxOn/LightBench (дата зверн. 27.10.2026).